CHƯƠNG 4 · BÀI 2

4.2. Các cơ chế thông tin liên lạc

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Nắm được các cơ chế liên lạc: tín hiệu, pipe, vùng nhớ chia sẻ, trao đổi thông điệp, socket.
- So sánh ưu nhược điểm của từng cơ chế.
- Biết khi nào nên dùng cơ chế nào.

4.2.1 Tín hiệu (Signal)

Giới thiệu: Tín hiệu là một cơ chế phần mềm tương tự như các ngắt cứng tác động đến các tiến trình. Một tín hiệu được sử dụng để thông báo cho tiến trình về một sự kiện nào đó xảy ra. Có nhiều tín hiệu được định nghĩa, mỗi một tín hiệu có một ý nghĩa tương ứng với một sự kiện đặc trưng.

Ví dụ : Một số tín hiệu của UNIX

Tín hiệu	Mô tả
SIGINT	Người dùng nhấn phím DEL để ngắt xử lý tiến trình
SIGQUIT	Yêu cầu thoát xử lý
SIGILL	Tiến trình xử lý một chỉ thị bất hợp lệ
SIGKILL	Yêu cầu kết thúc một tiến trình
SIGFPT	Lỗi floating – point xảy ra (chia cho 0)
SIGPIPE	Tiến trình ghi dữ liệu vào pipe mà không có reader
SIGSEGV	Tiến trình truy xuất đến một địa chỉ bất hợp lệ
SIGCLD	Tiến trình con kết thúc

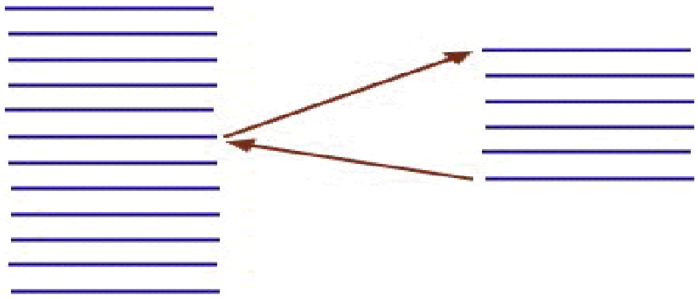
SIGUSR1	Tín hiệu 1 do người dùng định nghĩa
SIGUSR2	Tín hiệu 2 do người dùng định nghĩa

Mỗi tiến trình sỡ hữu một bảng biểu diễn các tín hiệu khác nhau. Với mỗi tín hiệu sẽ có tương ứng một trình xử lý tín hiệu (signal handler) qui định các xử lý của tiến trình khi nhận được tín hiệu tương ứng. Các tín hiệu được gởi đi bởi:

- Phần cứng (ví dụ lỗi do các phép tính số học)
- Hạt nhân hệ điều hành gởi đến một tiến trình (ví dụ lưu ý tiến trình khi có một thiết bị nhập/xuất tự do).
- Một tiến trình gởi đến một tiến trình khác (ví dụ tiến trình cha yêu cầu một tiến trình con kết thúc)
- Người dùng (ví dụ nhấn phím Ctl-C để ngắt xử lý của tiến trình)

Khi một tiến trình nhận một tín hiệu, nó có thể xử sự theo một trong các cách sau:

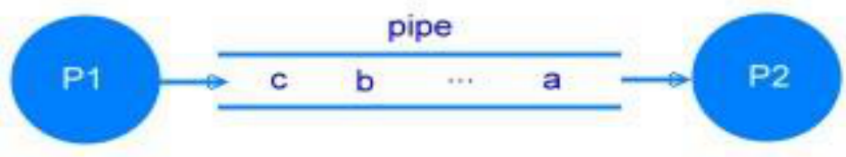
- Bỏ qua tín hiệu
- Xử lý tín hiệu theo kiểu mặc định
- Tiếp nhận tín hiệu và xử lý theo cách đặc biệt của tiến trình.



Hình 4.2.1-1. *Liên lạc bằng tín hiệu*

4.2.2 Pipe

Giới thiệu: Một pipe là một kênh liên lạc trực tiếp giữa hai tiến trình : dữ liệu xuất của tiến trình này được chuyển đến làm dữ liệu nhập cho tiến trình kia dưới dạng một dòng các byte. Khi một pipe được thiết lập giữa hai tiến trình, một trong chúng sẽ ghi dữ liệu vào pipe và tiến trình kia sẽ đọc dữ liệu từ pipe. Thứ tự dữ liệu truyền qua pipe được bảo toàn theo nguyên tắc FIFO. Một pipe có kích thước giới hạn (thường là 4096 ký tự)



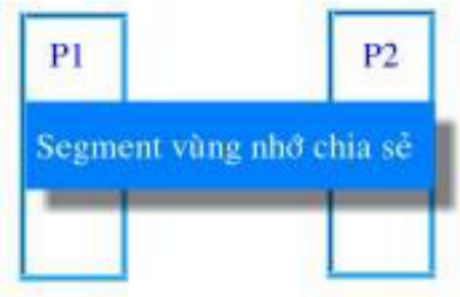
Hình 4.2.2-1*. Liên lạc qua pipe*

Một tiến trình chỉ có thể sử dụng một pipe do nó tạo ra hay kế thừa từ tiến trình cha. Hệ điều hành cung cấp các lời gọi hệ thống read/write cho các tiến trình thực hiện thao tác đọc/ghi dữ liệu trong pipe. Hệ điều hành cũng chịu trách nhiệm đồng bộ hóa việc truy xuất pipe trong các tình huống:

- Tiến trình đọc pipe sẽ bị khóa nếu pipe trống, nó sẽ phải đợi đến khi pipe có dữ liệu để truy xuất.
- Tiến trình ghi pipe sẽ bị khóa nếu pipe đầy, nó sẽ phải đợi đến khi pipe có chỗ trống để chứa dữ liệu. Đọc hoặc ghi, nhưng không thể thực hiện cả hai. Một số hệ điều hành cho phép thiết lập hai pipe giữa một cặp tiến trình để tạo liên lạc hai chiều. Trong những hệ thống đó, có nguy cơ xảy ra tình trạng tắc nghẽn (deadlock): một pipe bị giới hạn về kích thước, do vậy nếu cả hai pipe nối kết hai tiến trình đều đầy(hoặc đều trống) và cả hai tiến trình đều muốn ghi (hay đọc) dữ liệu vào pipe (mỗi tiến trình ghi dữ liệu vào một pipe), chúng sẽ cùng bị khóa và chờ lẫn nhau mãi mãi. Cơ chế này cho phép truyền dữ liệu với cách thức không cấu trúc. Ngoài ra, một giới hạn của hình thức liên lạc này là chỉ cho phép kết nối hai tiến trình có quan hệ cha-con, và trên cùng một máy tính.

4.2.3 Vùng nhớ chia sẻ

Giới thiệu: Cách tiếp cận của cơ chế này là cho nhiều tiến trình cùng truy xuất đến một vùng nhớ chung gọi là vùng nhớ chia sẻ (shared memory). Không có bất kỳ hành vi truyền dữ liệu nào cần phải thực hiện ở đây, dữ liệu chỉ đơn giản được đặt vào một vùng nhớ mà nhiều tiến trình có thể cùng truy cập được. Với phương thức này, các tiến trình chia sẻ một vùng nhớ vật lý thông qua trung gian không gian địa chỉ của chúng. Một vùng nhớ chia sẻ tồn tại độc lập với các tiến trình, và khi một tiến trình muốn truy xuất đến vùng nhớ này, tiến trình phải kết gắn vùng nhớ chung đó vào không gian địa chỉ riêng của từng tiến trình, và thao tác trên đó như một vùng nhớ riêng của mình.



***Hình 4.2.3-1**. Liên lạc qua vùng nhớ chia sẻ*

4.2.4 Trao đổi thông điệp (Message)

Giới thiệu: Hệ điều hành còn cung cấp một cơ chế liên lạc giữa các tiến trình không thông qua việc chia sẻ một tài nguyên chung , mà thông qua việc gởi thông điệp. Để hỗ trợ cơ chế liên lạc bằng thông điệp, hệ điều hành cung cấp các hàm IPC chuẩn (Interprocess communication), cơ bản là hai hàm:

- Send (message): gởi một thông điệp
- Receive (message): nhận một thông điệp

Nếu hai tiến trình P và Q muốn liên lạc với nhau, cần phải thiết lập một mối liên kết giữa hai tiến trình, sau đó P, Q sử dụng các hàm IPC thích hợp để trao đổi thông điệp, cuối cùng khi sự liên lạc chấm dứt mối liên kết giữa hai tiến trình sẽ bị hủy. Có nhiều cách thức để thực hiện sự liên kết giữa hai tiến trình và cài đặt các theo tác send /receive tương ứng: liên lạc trực tiếp hay gián tiếp, liên lạc đồng bộ hoặc không đồng bộ, kích thước thông điệp là cố định hay không … Nếu các tiến trình liên lạc theo kiểu liên kết tường minh, các hàm Send và Receive sẽ được cài đặt với tham số:

- Send(destination, message): gởi một thông điệp đến destination
- Receive(source,message): nhận một thông điệp từ source

4.2.5 Sockets

Giới thiệu: Một socket là một thiết bị truyền thông hai chiều tương tự như tập tin, chúng ta có thể đọc hay ghi lên nó, tuy nhiên mỗi socket là một thành phần trong một mối nối nào đó giữa các máy trên mạng máy tính và các thao tác đọc/ghi chính là sự trao đổi dữ liệu giữa các ứng dụng trên nhiều máy khác nhau. Sử dụng socket có thể mô phỏng hai phương thức liên lạc trong thực tế: liên lạc thư tín (socket đóng vai trò bưu cục) và liên lạc điện thoại (socket đóng vai trò tổng đài). Các thuộc tính của socket:

- Domaine: định nghĩa dạng thức địa chỉ và các nghi thức sử dụng. Có nhiều domaines, ví dụ UNIX, INTERNET, XEROX_NS, ...
- Type: định nghĩa các đặc điểm liên lạc:

  a) Sự tin cậy

  b) Sự bảo toàn thứ tự dữ liệu

  c) Lặp lại dữ liệu

  d) Chế độ nối kết

  e) Bảo toàn giới hạn thông điệp

  f) Khả năng gởi thông điệp khẩn

Để thực hiện liên lạc bằng socket, cần tiến hành các thao tác:

- Tạo lập hay mở một socket
- Gắn kết một socket với một địa chỉ
- Liên lạc: có hai kiểu liên lạc tùy thuộc vào chế độ nối kết:

a) Liên lạc trong chế độ không liên kết: liên lạc theo hình thức hộp thư:

- Hai tiến trình liên lạc với nhau không kết nối trực tiếp
- Mỗi thông điệp phải kèm theo địa chỉ người nhận.

Hình thức liên lạc này có đặc điểm được:

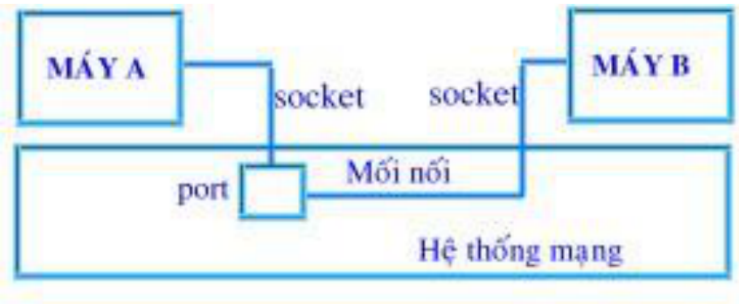
- Người gởi không chắc chắn thông điệp của học được gởi đến người nhận,
- Một thông điệp có thể được gởi nhiều lần,
- Hai thông điệp đượ gởi theo một thứ tự nào đó có thể đến tay người nhận theo một thứ tự khác.

Một tiến trình sau khi đã mở một socket có thể sử dụng nó để liên lạc với nhiều tiến trình khác nhau nhờ sử hai primitive send và receive.

b) Liên lạc trong chế độ nối kết: Một liên kết được thành lập giữa hai tiến trình. Trước khi mối liên kết này được thiết lập, một trong hai tiến trình phải đợi có một tiến trình khác yêu cầu kết nối.Có thể sử dụng socket để liên lạc theo mô hình client-serveur. Trong mô hình này, server sử dụng lời gọi hệ thống listen và accept để nối kết với client, sau đó, client và server có thể trao đổi thông tin bằng cách sử dụng các primitive send và receive.

Hủy một socket

Ví dụ: Trong nghi thức truyền thông TCP, mỗi mối nối giữa hai máy tính được xác định bởi một port, khái niệm port ở đây không phải là một cổng giao tiếp trên thiết bị vật lý mà chỉ là một khái niệm logic trong cách nhìn của người lập trình, mỗi port được tương ứng với một số nguyên dương.



***Hình 4.2.5-1**. Các socket và port trong mối nối TCP.*

Hình 4.2.5-1 minh họa một cách giao tiếp giữa hai máy tính trong nghi thức truyền thông TCP. Máy A tạo ra một socket và kết buộc (bind) socket nầy với một port X (tức là một số nguyên dương có ý nghĩa

cục bộ trong máy A), trong khi đó máy B tạo một socket khác và móc vào (connect) port X trong máy A.

Câu hỏi ôn tập

1. Tín hiệu (signal) là gì? Tiến trình có thể xử lý một tín hiệu theo những cách nào?
2. Trình bày cơ chế pipe và các hạn chế của nó.
3. So sánh vùng nhớ chia sẻ và trao đổi thông điệp.
4. Socket là gì? Vì sao socket phù hợp cho liên lạc giữa các tiến trình trên các máy khác nhau?

Bài 9: Các cơ chế thông tin liên lạc — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: gửi một tín hiệu không đồng nghĩa đã chuyển toàn bộ một bản ghi dữ liệu. Cơ chế thông báo sự kiện khác với giao thức truyền dữ liệu có cấu trúc; cần hiểu giới hạn và thứ tự xử lý của API cụ thể.

Tự luyện: mô tả gửi thông báo “có dữ liệu mới” và cách nhận dữ liệu thật từ hàng đợi. **Hướng giải:** bên nhận phải xử lý trường hợp trễ, đóng và lỗi; không làm thao tác phức tạp trong signal handler mà không kiểm tra quy tắc nền tảng. Xác định quyền và định danh tiến trình để tránh gửi nhầm.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/